

UNIVERSIDADE ESTADUAL DE MONTES CLAROS – UNIMONTES
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS – CCET
DEPARTAMENTO DE CIÊNCIAS DA COMPUTAÇÃO – DCC

EXERCÍCIO - CAPÍTULO II

RAMON LOPES DE QUEIROZ

MONTES CLAROS – MG
AGOSTO/2026

RAMON LOPES DE QUEIROZ

EXERCÍCIO - CAPITULO II

Atividade avaliativa apresentada para atendimento de requisito parcial para aprovação na disciplina Engenharia de Software do Curso de Graduação em Bacharelado em Sistemas de Informação – 3º período

Professor: Dra. Marilee Patta

MONTES CLAROS – MG

AGOSTO/2026

Capítulo 2

EXERCÍCIOS

1 Justificando sua resposta com base no tipo de sistema a ser desenvolvido, sugira o modelo genérico de processo de software mais adequado para ser usado como base para a gerência do desenvolvimento dos sistemas a seguir:

Um sistema para controlar o antibloqueio de frenagem de um carro.

Um sistema de realidade virtual para dar apoio à manutenção de software.

Um sistema de contabilidade para uma universidade, que substitua um sistema já existente.

Um sistema interativo de planejamento de viagens que ajude os usuários a planejar viagens com menor impacto ambiental

2 Explique por que o desenvolvimento incremental é o método mais eficaz para o desenvolvimento de sistemas de software de negócios. Por que esse modelo é menos adequado para a engenharia de sistemas de tempo real?

3 Considere o modelo de processo baseado em reuso. Explique por que, nesse processo, é essencial ter duas atividades distintas de engenharia de requisitos.

4 Por que é importante, no processo de engenharia de requisitos, fazer uma distinção entre desenvolvimento dos requisitos do usuário e desenvolvimento de requisitos de sistema

5 Descreva as principais atividades do processo de projeto de software e as saídas dessas atividades. Usando um diagrama, mostre as possíveis relações entre as saídas dessas atividades.

6 Por que, em sistemas complexos, as mudanças são inevitáveis. Exemplifique as atividades de processo de software que ajudam a prever as mudanças e fazer com que o software seja desenvolvido mais tolerante a mudanças (desconsidere prototipação e entrega incremental).

7 Por que os sistemas desenvolvidos como protótipos normalmente não devem ser usados como sistemas de produção.

8 Por que o modelo em espiral de Boehm é um modelo adaptável, que apoia tanto as atividades de prevenção de mudanças quanto as de tolerância a mudanças. Na prática, esse modelo não tem sido amplamente usado. Quais as possíveis razões para isso?

9 Quais são as vantagens de proporcionar visões estáticas e dinâmicas do processo de software, assim como no Rational Unified Process?

10 Historicamente, a introdução de tecnologia provocou mudanças profundas no mercado de trabalho e, pelo menos temporariamente, deixou muitas pessoas

desempregadas. A introdução da automação extensiva em processos pode vir a ter as mesmas consequências para os engenheiros de software? Justifique

Exercício - Capítulo 2, Engenharia de Software:

1 - Sistema de frenagem (ABS): Modelo em Cascata. Trata-se de um sistema crítico de tempo real que exige especificações exatas e rígidas antes de qualquer implementação.

Manutenção em realidade virtual: Desenvolvimento Incremental. Os requisitos de usabilidade e interação do sistema de realidade virtual são complexos e evoluem com validações práticas.

Contabilidade universitária: Orientado a Reúso. Sistemas de contabilidade que substituem plataformas existentes beneficiam-se da integração de pacotes comerciais (COTS) já consolidados no mercado.

Planejamento de viagens: Incremental. O desenvolvimento do sistema interativo requer ciclos curtos para adaptação aos feedbacks dos usuários.

2 - Sistemas corporativos operam em ambientes altamente dinâmicos. O desenvolvimento incremental permite ajustar a rota do projeto a cada ciclo, sem invalidar o trabalho já feito. Além disso, os usuários podem testar funcionalidades reais logo nas primeiras semanas de projeto. Isso garante que o produto final resolva o problema real do cliente, e não o que foi imaginado meses antes. Ainda, se um erro grave de concepção ocorrer, ele será descoberto cedo, custando muito menos para ser corrigido do que se fosse descoberto apenas na entrega final.

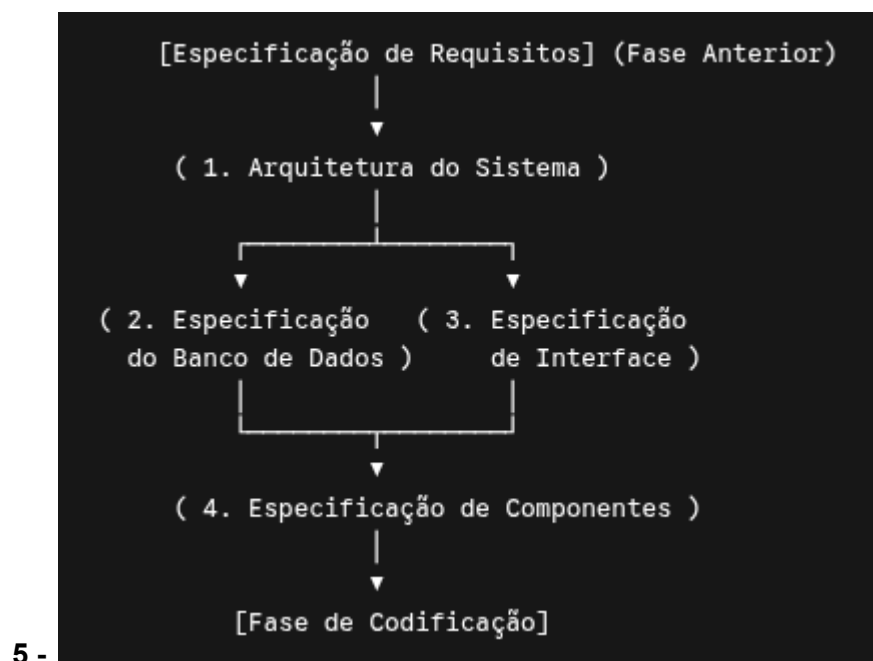
No entanto, se tratando de sistemas em tempo real, como os de controladores de voo de aviões, ele apresenta alguns problemas: Em sistemas de tempo real, o tempo de resposta é vital. O comportamento de um componente afeta o desempenho de todo o sistema. Muitas vezes, falhas nesses sistemas resultam em perda de vidas ou catástrofes financeiras. Além disso, softwares críticos precisam passar por rigorosos processos de certificação e necessitam de uma compatibilidade gigantes com todo o resto do sistema para funcionar devidamente.

3 - O primeiro levantamento de requisitos para definir o esboço, os objetivos, é o que permite iniciar a busca e análise de componentes. O esboço serve como uma wishlist e um filtro para avaliar o que o mercado. Após encontrar os componentes disponíveis, a equipe volta à mesa com o cliente para adaptar, alterar ou até descartar alguns dos requisitos iniciais com base no que os componentes selecionados realmente conseguem fazer. A ideia desse segundo levantamento é mostrar o quão próximo do comportamento desejado o projeto está, pois, por mais que nenhum sistema seja 100% idêntico ao escopo inicial, é

possível determinar uma porcentagem de aproximação após as mudanças, e vender esses valores aos clientes e realizar as mudanças devidas.

4 - Se uma equipe apresenta apenas requisitos de sistema para o cliente validar, ele se perderá na complexidade técnica e poderá aprovar um escopo que não resolve o problema real.

Em contrapartida, se você entregar apenas requisitos de usuário para a equipe de programação, a ambiguidade será imensa. Os desenvolvedores terão que adivinhar regras lógicas, limites de caracteres e padrões de segurança. Isso invariavelmente leva a erros estruturais graves que só são percebidos na fase final de testes.



6 - Em sistemas complexos, as mudanças são inevitáveis porque o ambiente no qual o software opera é altamente dinâmico. Regras de negócios, legislações tributárias e tecnologias subjacentes evoluem continuamente. Além disso, a própria compreensão do problema muda ao longo do tempo. É extremamente difícil para os clientes anteciparem todas as suas necessidades no início de um projeto.

Algumas coisas que ajudam a prever essas mudanças e adaptar o projeto são:

Gerenciamento de Requisitos e Rastreabilidade: Esta atividade ocorre em paralelo a todo o ciclo de vida e ajuda na previsão do impacto da mudança. Quando um cliente solicita uma alteração, a equipe não precisa adivinhar o impacto. A matriz mostra exatamente quais

arquivos, tabelas e funções precisarão ser reescritos, permitindo prever o custo e o tempo da mudança antes que ela ocorra.

Projeto Arquitetural focado em Baixo Acoplamento: Durante a fase de projeto de software, os arquitetos desenham o sistema dividindo-o em módulos ou componentes altamente independentes, utilizando princípios de abstração e encapsulamento. Isso torna o sistema tolerante à mudança. Se os componentes não dependem profundamente da lógica interna uns dos outros (baixo acoplamento), alterar a estrutura de dados de um módulo não causará falhas em cascata no resto do sistema. A mudança fica confinada a um espaço delimitado.

Engenharia de Testes Automatizados de Regressão: Durante e após a fase de codificação, a equipe desenvolve scripts de testes automatizados que validam todas as funcionalidades do sistema continuamente. A presença de uma suíte de testes robusta confere extrema tolerância à mudança. Os desenvolvedores podem modificar lógicas complexas ou substituir algoritmos inteiros com segurança. Se a alteração quebrar alguma funcionalidade antiga (regressão), os testes apontarão a falha imediatamente, impedindo que o erro se propague.

7 - Um protótipo tem um único objetivo principal na engenharia de software: validar requisitos, testar conceitos e reduzir incertezas rapidamente com o cliente. Na pressa de fazer a funcionalidade principal operar visualmente, a equipe ignora os atributos invisíveis, mas vitais, do software: Escalabilidade e desempenho, segurança, tratamento de segurança. Às vezes, para entregar resultados em dias ou semanas, os desenvolvedores ignoram as boas práticas de projeto, como o baixo acoplamento. O código costuma ser remendado, desorganizado e altamente interdependente. Tentar dar manutenção ou adicionar novas funcionalidades a essa base no futuro gerará um custo financeiro altíssimo. Também, um software de produção exige baterias de testes automatizados (para garantir que mudanças não quebrem o sistema) e documentação técnica mínima. O protótipo nasce sem nenhum dos dois. Se ele for para a produção e apresentar um erro grave, a equipe não terá como rastrear a falha rapidamente.

8 - Apesar de sua elegância teórica e superioridade técnica na gestão de riscos, o modelo em espiral encontrou severas barreiras no mundo corporativo diário:

Exigência de Alta Especialização: O sucesso do modelo depende quase que inteiramente da habilidade da equipe em identificar, quantificar e mitigar riscos em *cada* ciclo. Profissionais com essa expertise sênior são raros e caros; uma análise de riscos malfeita faz o projeto inteiro desmoronar.

Custo e Burocracia Excessivos: A estrutura de planejamento, análise de riscos e revisão após cada loop adiciona uma sobrecarga administrativa imensa. Para projetos de pequeno e médio porte, o custo de gerenciar a espiral frequentemente ultrapassa o custo de simplesmente programar o sistema.

Conflito com Contratos Comerciais: A maioria das empresas de software trabalha com contratos de escopo ou orçamento fixos. É extremamente difícil vender um projeto para um cliente dizendo que o software evoluirá em espirais indefinidas e que ele pagará por "fases de análise de risco".

Ascensão do Agile: Metodologias ágeis (como o Scrum) surgiram oferecendo os mesmos benefícios iterativos (tolerância a mudanças e entregas em ciclos curtos) de uma forma muito mais simples, enxuta e focada na entrega de valor ao invés de documentação de riscos, dominando o mercado.

9 - Ao proporcionar ambas as visões simultaneamente, o modelo oferece benefícios diretos para a engenharia de software:

Desvinculação entre Fase e Disciplina: No modelo em cascata, a fase de "Testes" só ocorre após a "Implementação". O RUP mostra que isso é irreal. Através do gráfico bidimensional, a equipe visualiza que a disciplina de *Testes* ocorre em todas as fases do projeto (dinâmico), embora seu pico de intensidade seja na fase de Construção.

Alocação Precisa de Recursos: A visão cruzada permite que o gerente de projetos saiba exatamente quem alocar e quando. Na fase de Concepção, há um pico na disciplina de *Requisitos*, exigindo muitos Analistas de Negócios e poucos Programadores. Na Construção, a curva se inverte.

Apoio à Iteração: A visão dinâmica mostra que o tempo avança em iterações (pequenos ciclos). A visão estática garante que, dentro de cada uma dessas pequenas iterações, a equipe execute um pouco de cada disciplina (levantar um requisito, projetar a solução, programar e testar).

Flexibilidade e Customização: Como as atividades (estáticas) estão separadas do tempo (dinâmico), uma empresa pode customizar o processo facilmente. Se for um projeto de alto risco, pode-se "esticar" a fase de Elaboração no eixo dinâmico, sem alterar as regras técnicas das disciplinas no eixo estático.

10 - O engenheiro de software será afetado pela automação extensiva. Tem se tornado cada vez menos necessário decorar linhas e linhas de códigos, e realizar processos

repetitivos na produção do software. Com isso, essas tarefas repetitivas podem ser atribuídas para a máquina e permite o engenheiro focar em outras partes que outrora seriam difíceis de se realizar enquanto se produz o código. No entanto, diversos paradigmas da programação tem se tornado cada vez mais obsoletos, se tornando necessário adaptar as novas tecnologias e forçando os engenheiros e constantemente estudarem esses novos funcionamentos, para não se tornarem obsoletos também. O ideal para esses engenheiros é: entender o que o cliente quer, organizar a arquitetura com a IA implementada, e sempre revisar o código gerar e garantir o bom funcionamento do sistema.